IG AGENT

v25 — Complete Final Specification

The definitive north star for the entire v25 build

Points Reinforcement | Three-Speed Learning | Web Dashboard | ATR Exits | 8 Weeks to Live

Version	Foundation	Date	Target	Status
v25-dev	**v24-proven**	**May 2026**	**8 Weeks**	**FINAL v5**

Purpose — Read This First

This is the complete, final specification for IG Agent v25. It incorporates every design decision, every lesson learned from v24's proven trading sessions on 25 May 2026, and every enhancement identified. v25 is a step change across three dimensions: (1) a complete GUI rebuild from Tkinter to a web-based dashboard, (2) a full AI/ML learning layer with points-based reinforcement, and (3) a production-grade trading engine with all operational fixes proven in v24. This document is Cursor's north star. Nothing changes after this.

1. What v25 Is — The Step Change

v25 is Three Step Changes in One

STEP 1: GUI — Replace Tkinter with a web-based dashboard. No more beach balls, no more macOS rendering issues. Professional real-time trading interface accessible from any device. STEP 2: AI/ML — Points-based reinforcement learning that scores every trade, learns from wins and losses, and continuously improves signal quality. STEP 3: Production Trading Engine — All operational fixes proven on 25 May 2026 built in from day one: Lightstreamer, async orders, session management, data integrity.

Dimension	v24 Proven	v25 Target
GUI Framework	Tkinter — beach ball on every tab	Web dashboard — React + FastAPI, no freeze ever
Stream	Lightstreamer (fixed 25/05)	Lightstreamer + WebSocket to dashboard
Signal gate	Confidence >= 92 only	Conf + ML + fitness + points — all agree
Exit management	Fixed stop + fixed target	ATR trail + break-even + partial close
Learning	None	Points reinforcement + nightly 100x replay
Trade data integrity	Phantom trades, wrong currency (fixed 25/05)	IG-confirmed only, GBP always, pending state
Session management	Manual restart at market open (fixed 25/05)	Automatic session refresh, gap protection
Order execution	Synchronous — blocked loop (fixed 25/05)	Async — loop never blocked
Position sync	Stopped on watchdog kill (fixed 25/05)	Independent of loop lifecycle
Watchdog	Killed at 60s during order confirm (fixed 25/05)	300s + 3 auto-restarts
REST budget	Saturated at open (fixed 25/05)	Protected at open, reserved during orders
Deal reconciliation	Open/close ID mismatch (fixed 25/05)	Dual ID matching, auto-reconciliation script
Win rate target	55% all-time (proven)	60%+ consistently
Monitoring	Tab-based, beach ball, manual	Real-time web dashboard, mobile accessible

2. Lessons Learned from v24 — Built Into v25 From Day One

What 25 May 2026 Taught Us

Every fix made today exposed a design decision that v25 must get right from the start. This section documents what was learned and how v25 addresses each lesson permanently.

Lesson	v24 Problem	v25 Solution
Lightstreamer auth	PROXY_SERVER adapter rejected; CST as username instead of account_id	adapter=None, setUser(account_id), CST-|XST- password — correct from day one
Market open session	Afternoon session carried to evening; REST saturated; stream got zero ticks	Auto session refresh on market open, 120s REST pause, stream gets priority
Async orders	Synchronous REST order blocked the trading loop for 20-40s	All orders async via background worker thread — loop never blocked
Watchdog false kills	65s threshold killed loop during order confirm	300s tick-count watchdog + 3 auto-restarts
Position sync lifecycle	Stopped when loop died; ghost positions overnight	Position sync independent of loop — always runs while DEMO session active
Quote freshness	LS CONNECTED but hub stale; _next_quote skipped REST fallback	Direct hub.publish() in onItemUpdate; REST fallback when age > 35s regardless
Deal ID mismatch	Open deal ID (DIAAA) never matched close ref (LBH6) in txn sync	Store both IDs; reconcile via dual lookup; auto-reconciliation script
Currency display	pnl_points shown as $USD instead of GBP	Account currency from IG session always; pts shown as pts when unconfirmed
Phantom trades	Local position-sync closes shown as confirmed before IG confirms	Pending state until IG transaction confirms; never show WIN/LOSS until confirmed
SIM/soak pollution	179 SIM trades could leak into main display	EXCLUDED_SOURCES permanent filter; SIM never shown in main GUI

LiveTradeGate	5 arming ticks blocked first signal; WAIT->BUY edge requirement	2 arming ticks; any validated signal after arming passes — no edge requirement
GUI beach ball	Tkinter renders lifecycle diagram on main thread; 3-4s freeze	Web dashboard — React renders in browser; zero main thread blocking ever
Transaction sync at open	force=True startup sync saturated REST budget	120s defer when market open at launch; paused during order in flight
Signal threshold	92 too tight for proof; 90 used in practice	Review threshold after 20 sessions; document in config clearly

3. Three-Speed Learning System

Nightly Replay Accelerates Everything

Without replay: Demo accumulates 200-300 labels over 8 weeks. With replay: first overnight run generates 2,000+ labels from 2 years of real Nikkei history. By Week 4 the model has 8,000+ labels. The agent arrives at Demo already trained. Demo validates — it does not wait for data.

Speed	Mode	Purpose
100x — Nightly replay	TEST — no IG connection	Bulk training from 2 years real Nikkei 5m/15m history. Runs nightly 06:15-22:30 BST. Generates thousands of labelled decisions overnight. Model improves every night.
1x — Shadow	DEMO — live stream, no gating	Validates replay model on real live ticks. Logs what ML would decide alongside rules-only trading. Confirms model generalises beyond history.
1x — Live trading	DEMO/LIVE — real trades	Ground-truth labels — 3x weight. Every real trade outcome feeds back. Continuously improves model quality.

3.1 Nightly Replay Cycle

Time	Action
06:00 BST	Japan 225 session closes — session end flatten runs
06:15 BST	Replay scheduler starts automatically
06:15-08:00	Pull new IG bars, run 100x replay, extract labels, retrain model
08:00-09:00	Evaluate vs current model; stage for approval if better
22:30 BST HARD CUTOFF	Replay stops — live trading gets full REST budget
23:11 BST	Japan 225 opens — agent trades with latest approved model

3.2 Label Weighting

Source	Weight	Rationale
Real DEMO trade WIN/LOSS	3x	Ground truth — agent actually executed
Replay label WIN/LOSS	1x	Simulated on real history — good signal, lower trust
Shadow label (would-have)	0.5x	Indirect inference — ML predicted, not confirmed

4. Points-Based Reinforcement System

Every Trade Scores or Loses Points

Points are confidence-weighted — a high-confidence loss is the most damaging event. Points drive position sizing, trade frequency, and automatic backing-off. The agent learns that confidence must be earned.

4.1 Scoring Formula

Event	Points
WIN — high conviction (ML >= 0.85)	+3 x (actual_pnl / avg_win_pnl)
WIN — standard (ML 0.65-0.85)	+2 x (actual_pnl / avg_win_pnl)
WIN — marginal (ML at threshold)	+1 flat
LOSS — high conviction (ML >= 0.85)	-4 x (actual_loss / avg_loss) — LARGEST PENALTY
LOSS — standard (ML 0.65-0.85)	-2 x (actual_loss / avg_loss)
LOSS — marginal	-1 flat
BREAKEVEN	0
Partial close — banked portion	Scored immediately on banked P&L
Trail exit better than fixed would have been	+0.5 bonus
ML suppressed — would have lost	+0.5 (capital protected)
ML suppressed — would have won	-0.5 (missed opportunity)

4.2 Staged Backing-Off and Recovery

State	Cumulative Points	Behaviour	Recovery
HEALTHY	> +10	Full size, standard threshold, max 6 positions	N/A
CAUTION	-5 to +10	0.75x size, threshold +2, max 3 positions	Return to +10 OR 5 wins
WARNING	-15 to -5	0.5x size, threshold +5, max 2 positions	Return to -5 OR 3 wins

DANGER	-30 to -15	0.25x size, threshold +10, max 1 position	Return to -15 OR 3 wins
STOP	< -30	No new trades — manual approval required	Manual only
SESSION PAUSE	3 consec losses	Skip next 3 signals	After 3 skips
DAY STOP	Session score < -5	No more trades today	Next session open

5. Exit Management — ATR Trailing Stops

Exits Are As Important As Entries

Fixed stops cap upside and ignore momentum. ATR trailing stops let winners run while protecting capital. Partial closes bank profit while holding a runner. This directly grows the points score by maximising winning trade value.

Exit Mechanism	Logic
Break-even trigger	Trade reaches +1.0x ATR profit → stop moves to entry price
Trail activation	Immediately after break-even — follows price in profit direction only
Trail distance — high ML (>= 0.85)	1.75x ATR — high conviction gets more room to run
Trail distance — standard	1.5x ATR
Trail distance — low ML (< 0.70)	1.0x ATR — tighter on weaker signals
Partial close trigger	Trade reaches +1.5x ATR profit → close 50%, score points immediately
Momentum tighten	RSI diverges against trade → trail tightens to 0.75x ATR
Time-based exit	Open > 6 bars AND price moved < 0.3x ATR → exit at market
Maximum cap	3.0x ATR profit — hard exit regardless of trail
Re-entry after trail	15m trend still strong → re-entry eligible at threshold +3, 0.75x size
Short trades	Identical logic — mirrored direction
Iron rule	Trail ONLY moves in profit direction — never backwards

6. Session Management — Proven Fixes Built In

Every Session Management Fix from 25 May is Standard in v25

The session lifecycle failures that prevented trades on 24 May and caused problems on 25 May are designed out of v25. These are not fixes — they are the correct architecture from day one.

Requirement	Implementation
Auto session refresh on market open	Detect closed->open transition; stop stale session; start fresh DEMO automatically
REST budget protected at open	120s pause on background REST; stream and stream probe get priority
Transaction sync deferred at open	force=True startup sync waits 120s when market already open at launch
REST budget reserved during orders	begin_order_in_flight() pauses non-essential REST until confirm returns
Session end exit (mandatory)	All positions closed T-5min before session end — automatic, no exceptions
Gap open protection	Fitness capped at 40 for first 15min after gap > 1.0x ATR
Cold start protection	Fitness capped for first 6 bars — agent needs context before signals
DST-aware session times	Session times from IG market details API — never hardcoded
State persistence	All component state written to disk atomically — survives restarts
Position sync independent	Runs while DEMO session active regardless of loop state
Watchdog 300s + auto-restart	3 auto-restart attempts before alerting; never kills during order confirm

7. Environment Fitness System

Factor	Weight	Scoring
ATR vs 20-period rolling average	30%	< 1.2x = 100pts / > 1.8x = 0pts
15m trend strength	25%	Strong aligned = 100pts / Flat = 0pts
Session timing	20%	First 2hrs = 100pts / Final 30min = 20pts
Spread vs normal	15%	< 1.3x = 100pts / > 2x = 0pts
Recent signal quality	10%	Last 5 clean = 100pts / Choppy = 0pts

Score Range	Behaviour
80-100 Excellent	Trade at full size
60-79 Good	Trade at standard size
40-59 Marginal	Trade at 0.75x size, ML threshold +2
20-39 Poor	WAIT — log reason
0-19 Hostile	WAIT — flag for review

8. Web Dashboard — Complete GUI Rebuild

Why Tkinter Must Be Replaced

v24 proved that Tkinter is fundamentally unsuitable for a real-time trading agent on macOS. The beach ball (main thread blocking) occurs whenever any GUI component renders while trading logic runs. This cannot be fixed within Tkinter — it requires a different architecture. v25 replaces Tkinter entirely with a web-based dashboard that has zero main thread blocking by design.

8.1 Technology Stack

Component	Technology
Backend API	FastAPI (Python) — same language as trading engine
Real-time data	WebSocket — push prices, trades, signals to browser instantly
Frontend	React — component-based, high-performance rendering
Charts	Recharts or TradingView Lightweight Charts — professional financial charts
Styling	Tailwind CSS — dark professional theme
State management	React hooks — clean real-time state updates
Mobile	Responsive design — works on phone and tablet
Alerts	Browser notifications + sound — no desktop popup needed
Local access	http://localhost:8080 — opens in any browser
Remote access	Optional: ngrok or Tailscale for secure remote access from phone

8.2 Why This Stack

Requirement	How Web Stack Delivers
No beach ball	Browser renders in its own process — trading engine never blocked
Real-time prices	WebSocket pushes every Lightstreamer tick to browser instantly
Professional look	React + Tailwind = Bloomberg-quality dark theme
Mobile access	Responsive CSS — check trades from phone while away from Mac
Performance	React virtual DOM — only changed elements re-render

Charts	TradingView-quality P&L curves and price charts built in
Zero Tkinter	Trading engine has no GUI dependency — cleaner, faster, testable
Future alerts	Push notifications to phone via browser notification API

8.3 Dashboard Layout — Five Panels

Design Principle: Professional, Simple, Uncluttered

The dashboard has five main views. Each is a single page with no tabs to click between. Real-time data flows continuously. Everything visible at a glance. Dark professional theme matching trading platform aesthetics.

Panel	Contents
LIVE — Primary view	Large bid/offer price, IG STREAM status, agent state banner, active trade panel, Why No Trade, last signal confidence gauge
TRADES — Active and closed	Active positions with live P&L updating, stop/target levels, trail status, partial close status. Closed trades in GBP with WIN/LOSS/Pending. IG-confirmed only.
POINTS — Performance	Live trade/session/cumulative points score, agent state (HEALTHY/ CAUTION/WARNING/DANGER/STOP), win rate chart, P&L curve, benchmark comparisons
INTELLIGENCE — Weekly report	Trade autopsy summaries, pattern library, what worked/failed this week, model performance, parameter recommendations
SYSTEM — Operations	REST budget chart, stream health, position sync status, model version, last retrain, E2E test runner, emergency stop button

8.4 Real-Time Barometers — Always Visible Header

Barometer	Shows	Updates
BID / OFFER	Live Japan 225 price — large, prominent	Every Lightstreamer tick
Agent State	HEALTHY / CAUTION / WARNING / DANGER / STOP	On points change
Points: Trade	Last trade score — green/red	After each close
Points: Session	Today running total	After each close
Points: Cumulative	All-time master score	After each close
ML Confidence	Current ML score (0-1) gauge	Every signal

Signal Strength	Current confidence (0-99) gauge	Every tick
Environment Fitness	Fitness score + regime label	Every 5m bar
Win Rate Today	Won/total today %	After each close
Win Rate All Time	Running DEMO win rate	After each close
Daily P&L	Running GBP P&L	Every position update
Stream Status	LIVE / STALE / DISCONNECTED	Every tick
REST Budget	X/6 calls/min	Every REST call

8.5 Active Trade Display

Per-Position Real-Time Panel

For each open position: entry price, current price (live), unrealised P&L in GBP (live), trail stop level (updates every bar), break-even status, partial close status, ML score at entry, fitness score at entry, time open, distance to stop (pts), distance to target (pts). No refresh needed — WebSocket pushes every update.

8.6 ML Decision Log — Every Signal Explained

Nothing is a Black Box

Scrolling log of every signal evaluation: [time] Conf:94 ML:0.73 Fit:81 Points:HEALTHY -> TRADE 1.0x. Or: [time] Conf:93 ML:0.47 Fit:65 Points:CAUTION -> SUPPRESSED (ML below threshold). Agent always explains its reasoning. Accessible from any device.

8.7 Sound Alerts

Event	Alert
Trade opened	Single tone
Trade closed — WIN	Double positive tone
Trade closed — LOSS	Single low tone
Partial close executed	Brief tone
Agent state change (caution/warning)	Double amber tone
STOP state activated	Alarm — high priority
20% drawdown hit	Alarm — high priority

Fail safe activated	Alarm — high priority
New model ready for approval	Single tone
Session end approaching (15min)	Reminder tone
Stream STALE > 60s	Medium priority tone

8.8 Migration from Tkinter

Migration Step	Detail
Phase 1: Backend API first	FastAPI server exposes all bot_controller data via REST + WebSocket. Tkinter GUI still runs in parallel. Zero trading code changes.
Phase 2: React dashboard	Build React frontend connecting to FastAPI. Run side by side with Tkinter during validation.
Phase 3: Tkinter removed	Once web dashboard proven over 1 week of live trading, remove Tkinter entirely. Agent starts as headless process + web server.
Rollback plan	Tkinter code stays in git history. One commit reverts if needed.
No trading code changes	FastAPI reads bot_controller state — it never writes to trading path

9. Trade Intelligence Engine

The Agent Learns Why — Not Just What

Every closed trade is automatically analysed. Winning and failing patterns are extracted and fed back into the model weekly. The knowledge base grows every session. High-confidence losses are investigated most deeply.

9.1 Trade Autopsy — Every Closed Trade

Field Analysed	What It Reveals
confidence_at_entry vs result	Was confidence justified? High conf + loss = model weakness
ml_score_at_entry vs result	Was ML score predictive? Calibrates model accuracy
fitness_score_at_entry vs result	Did environment score predict outcome?
atr_at_entry vs result	Does high ATR reliably predict failure?
trail_exit vs fixed_target	Did trailing stop improve P&L vs fixed target?
partial_close vs hold_all	Did partial close improve total P&L?
session_time vs result	Which session windows are most reliable?
points_state_at_entry vs result	Do we trade worse in caution/warning state?
regime_at_entry vs result	Which regimes produce best results?
replay_prediction vs actual	Did replay training predict this outcome? Calibration check

9.2 Data Integrity — v24 Lessons Applied

Rule	Implementation
IG-confirmed only in main display	Pending state until ig_pnl_currency set by txn sync
GBP always	Account currency from IG session; never instrument config currency
Dual deal ID reconciliation	Store open_deal_id AND ig_close_deal_id; match via both
SIM/soak permanently excluded	EXCLUDED_SOURCES constant; never shown in main GUI
Auto-reconciliation script	scripts/reconcile_pending_trades.py — runs after session
Autopsy only on confirmed trades	Never analyse phantom or pending trades — corrupts model

10. Complete System Architecture

10.1 All Components

Component	Responsibility
IG REST / Lightstreamer	Live tick reception, order routing, position sync — v24 proven
FastAPI server (NEW)	Exposes all bot state via REST + WebSocket to web dashboard
Web Dashboard (NEW)	React frontend — real-time barometers, trades, points, intelligence
Replay Scheduler (NEW)	Nightly 06:15-22:30 BST — pulls bars, runs 100x replay, retrains
Replay Engine (NEW)	Feeds historic bars through full agent at 100x — extends fast_replay.py
State Manager (NEW)	Persists all component state atomically — survives restarts
Session Manager (NEW)	Auto open/close handling, gap protection, cold start, DST times
Environment Scorer (NEW)	5-factor fitness + regime + historical context
Points Engine (NEW)	Trade/session/cumulative scoring, backing-off, recovery
ML Scorer (NEW)	XGBoost via subprocess — scores signal at gate time
Trade Manager (NEW)	ATR trail, break-even, partial close, momentum exit, re-entry
Correlation Guard (NEW)	Direction limits, confidence decay, intra-day win rate protection
Trade Autopsy (NEW)	Analyses every confirmed close — pattern extraction
Insight Store (NEW)	Pattern library — winning and failure conditions
Benchmarking (NEW)	5 continuous benchmarks — proves AI adding value
Reconciliation Script	scripts/reconcile_pending_trades.py — runs after each session
Emergency Stop (NEW)	scripts/emergency_stop.sh — closes all + locks agent

10.2 The 13-Gate Decision Flow

Gate	Pass Condition	Fail Action
1. Market open	Session hours OK, weekend block clear	WAIT
2. Cold start	6+ complete bars since session open	WAIT — log bars remaining
3. Gap check	No extreme gap — or wait period elapsed	WAIT — log gap size

4. Environment fitness	Score >= 40	WAIT — log score and factors
5. Points state	Not STOP, not session/day pause	WAIT — log points state
6. Correlation guard	Same-direction < 2, signal not decayed	WAIT — log reason
7. Signal confidence	Confidence >= threshold (92 production)	WAIT — normal
8. Risk validation	Spread, ATR, position limits, GBP 100 cap	WAIT — v24 rules
9. ML gate	ML score >= ml_threshold	WAIT — log ML score
10. Size calculation	Points + ML + fitness -> multiplier	Default 0.5x if error
11. Execute (async)	IG order background worker — loop continues	Log failure
12. Trail monitoring	Updates every 5m bar, partial close at 1.5x	Log if fails
13. Exit / session end	Trail, time, momentum, or session end flatten	Force close at T-5

11. Eight-Week Delivery Plan

Replay Acceleration Makes 8 Weeks Achievable

First nightly replay run generates 2,000+ labels. By Week 4: 8,000+ labels. Agent arrives at Demo already trained. Demo validates — not waits for data.

S1 **Week 1**	**Core AI + Session Engine** **Goal: Agent trading in Demo with ML gate, points, ATR trail, session safety**	**Gate:** 14/14 pass + soak pass + 3 stable Demo sessions + points not in DANGER
BUILD	• state_manager.py — atomic state persistence, restart recovery • points_engine.py — full 3-level scoring, backing-off, recovery • environment_scorer.py — 5-factor fitness, regime detection • trade_manager.py — break-even, ATR trail, partial close, re-entry • session_manager.py — auto market open refresh, gap, cold start, DST • replay_engine.py — extends fast_replay.py, 100x historic bars • replay_scheduler.py — nightly automation with 22:30 BST cutoff • emergency_stop.sh — closes all positions, locks agent • Pull IG REST Nikkei history, run first overnight replay (~2,000 labels) • Wire all 13 gates in trading_loop.py • Config validation on startup — safe defaults for all new keys • FastAPI server skeleton — WebSocket endpoint, health check	
MONITOR	• Win rate improving vs v24 baseline (55%)? • Points trending up across sessions? • ATR trails generating better exits than fixed stops? • Replay accuracy within 10% of real Demo outcomes? • State persistence: restart mid-session, confirm full recovery • Emergency stop: run script, confirm all positions close	

S2 **Week 2**	**Trade Intelligence + Learning** **Goal: Agent learns from every trade — patterns emerging**	**Gate:** Patterns identified + weekly report live + 14/14 pass

BUILD	• trade_autopsy.py — full autopsy on every confirmed close • insight_store.py — winning/failure pattern library • environment_scorer.py — wire historical context from insight store • reconcile_pending_trades.py — scheduled post-session run • Weekly intelligence report v1 — what worked, what failed • Replay enhanced with autopsy pattern features (~4,000 labels) • Data integrity: all autopsy only on IG-confirmed trades
MONITOR	• Meaningful patterns emerging from autopsy data? • Fitness score more accurate with historical context? • Weekly report delivering actionable insights? • Zero phantom trades in closed trades display?

S3 Week 3	**Correlation and Protection** **Goal: No correlated losses, full regime awareness**	**Gate:** Zero correlated loss events + regime detection validated + 14/14 pass
BUILD	• correlation_guard.py — direction limits, confidence decay, intra-day protection • session_manager.py — enhanced gap detection, cold start guard • Full regime detection — TRENDING/CHOPPY/VOLATILE/RANGING • Intra-day win rate protection — session stop if win rate < 30% after 5 trades • Nightly replay — ~6,000 labels, regime-aware model	
MONITOR	• Zero 3x same-direction losses in a week? • Cold start protection reducing early session loss rate? • Regime detection accurate vs visible market behaviour?	

S4 Week 4	**Web Dashboard** **Goal: Complete professional GUI — no beach ball ever**	**Gate:** Web dashboard live + no performance issues + 14/14 pass
BUILD	• FastAPI server — all bot state via REST + WebSocket • React dashboard — LIVE panel with real-time prices • React dashboard — TRADES panel with active/closed positions • React dashboard — POINTS panel with barometers and P&L chart • React dashboard — INTELLIGENCE panel with weekly report • React dashboard — SYSTEM panel with operations chart • All 13 barometers updating via WebSocket • ML decision log scrolling in LIVE panel • Sound alerts via browser Web Audio API • Mobile-responsive design — works on phone • Tkinter still running in parallel during validation • Nightly replay — ~8,000 labels, richest pre-Demo model	

MONITOR	• No beach ball on any panel switch? • WebSocket prices updating in < 1s? • All barometers accurate vs engine.log? • Works correctly on mobile browser? • Tkinter and web dashboard showing identical data?

Weeks 1-4 Complete — Agent Fully Built, Enters Demo Monitoring

By end of Week 4: all components built. Model trained on 8,000+ replay labels. Professional web dashboard live. Tkinter removed. Demo monitoring begins Week 5.

DEMO **Weeks 5-8 — Monitoring and Validation**	• Agent trades every Japan 225 session — approximately 20 sessions over 4 weeks • Nightly replay continues — real Demo labels grow and increase in weight (3x) • Weekly intelligence report reviewed every Sunday before market open • Reconciliation script runs after every session — zero phantom trades • Model retrains weekly with your approval — real labels dominating by Week 6 • Week 6 mid-review: win rate trend, points trajectory, benchmark status • New model staged for approval — GUI notification if unapproved > 24hrs • Auto-rollback: win rate < 50% for 3 days -> revert to previous model • Week 8 decision: all live gates passed? Proceed to Live. Not passed? Extend 2 weeks. • NEVER rush Live — real money requires real confidence from real Demo evidence

12. Monitoring Plan

12.1 Daily (5 minutes — web dashboard on any device)

Check	Where
Any trades last session?	TRADES panel — closed trades list
Points state healthy?	POINTS panel — cumulative score
Replay ran overnight?	SYSTEM panel — last replay timestamp
New model staged?	SYSTEM panel — model version + approval notification
Any errors?	SYSTEM panel — error count + engine.log link
Stream healthy?	LIVE panel — stream status indicator
Session end flatten worked?	TRADES panel — zero open positions after 06:00 BST

12.2 Weekly (30 minutes — Sunday before 23:11 BST)

Review	What to Look For
Intelligence report	Patterns, failures, what to watch next week
Win rate trend	Above or below 60% target? Improving?
Points trajectory	Cumulative trending up? Consistently HEALTHY?
Trail performance	Adding P&L vs fixed target benchmark?
Replay accuracy	Within 10% of real Demo outcomes?
Model approval	Review comparison — approve or reject new model?
Live gate status	How many of 16 items satisfied?

13. Live Trading Gate — 16 Hard Gates

Every Item Must Be Satisfied

This is not advisory. Every item below must be confirmed before allow_live_trading is set to true.

Gate	Target
Demo trades	200+ complete cycles (open to close)
Win rate	60%+ over last 100 Demo trades consistently
Cumulative points	HEALTHY for 2+ consecutive weeks
Demo sessions	15+ sessions completed and logged
Open market proof	PASSED (v24 proven 25/05/2026)
ML contribution	ML adding > 5% win rate vs rules-only
Trail performance	ATR trails demonstrably improving P&L
Replay calibration	Within 10% of actual Demo performance
Trade autopsy	Running on every confirmed close — no gaps
Weekly retrain	Approval workflow tested 2+ complete cycles
Protective stops	Always sent — zero exceptions
Session end flatten	Zero positions held through session end
Emergency stop	Script tested — closes all and locks
State persistence	Restart mid-session tested — full recovery
Infrastructure	caffeinate + launchd stable 2+ weeks
Human sign-off	You are personally satisfied — judgement overrides checklist

14. Component Safety and Performance

The Trading Loop Must Never Be Blocked

Every component implements fail-safe: try-catch, safe default return, log warning, never crash. The web dashboard runs in a separate process — it cannot block trading under any circumstances.

14.1 Safe Defaults

Component	Safe Default on Failure
environment_scorer.score()	Return 50 — neutral, trading continues
points_engine.get_state()	Return HEALTHY — never block on points error
points_engine.get_size_multiplier()	Return 0.5x — conservative but allows trading
ml_scorer.score()	Return 0.0 — treat as suppressed, log SAFE MODE
trade_manager.update_trail()	Keep existing stop — never remove on error
session_manager.check_session_end()	Return False — safer than false positive close
FastAPI server failure	Trading continues headless — web dashboard reconnects
Replay scheduler failure	Log error, skip night — next night retries

14.2 Performance Budget

Component	Max Time — Trading Path
All trading gates combined	< 600ms per tick — well within 5s loop
ML scorer subprocess	< 300ms — monitored
FastAPI WebSocket push	Async — never in trading path
Web dashboard render	In browser process — zero impact on trading
Replay (nightly)	Separate process — hard cutoff 22:30 BST
Trade autopsy	Runs post-close — never in tick loop

15. Operational Safety

15.1 State Files — Restart Recovery

File	Contents	Updated
src/data/state/points_state.json	Points at all 3 levels, state, consecutive counts	After every trade
src/data/state/trail_state.json	Trail level, break-even, partial close per position	Every 5m bar
src/data/state/session_state.json	Session open time, bars elapsed, gap detected	Every 30s
src/data/state/replay_state.json	Last replay timestamp, bar count, calibration factor	After each replay

15.2 Emergency Stop

Step	Action
Run	bash scripts/emergency_stop.sh
Step 1	Close all open IG positions via REST API
Step 2	Set allow_live_trading = false in config
Step 3	Kill all agent processes
Step 4	Write emergency_stop.lock to project root
Recovery	Delete lock file manually, review config, restart
On startup with lock	Agent refuses to start — requires manual deletion

15.3 Multi-Instance Protection

Agent Must Never Run as Two Instances

On startup: check lock file AND running processes by pattern. If live instance detected: log error and refuse to start. launchd auto-restart increases this risk — protection is mandatory.

16. Multi-Instrument Architecture

v25 trades 4 instruments simultaneously with a single position slot

EUR/USD, Japan 225, Wall Street, and Gold all run independent signal engines in parallel. Each instrument watches ticks, generates signals, and evaluates confidence against its own threshold. Only 1 position may be open at any time across all instruments combined — whichever instrument fires first wins the slot. All others wait until that position closes. This gives maximum signal coverage with zero overexposure.

16.1 Instrument Registry

EUR/USD	CS.D.EURUSD.CFD.IP	88%	London + US overlap 12:00–17:00 BST	3 pips	FX, 24hr, tight spreads, high liquidity
Japan 225	IX.D.NIKKEI.IFM.IP	92%	Tokyo session 23:00–06:00 BST	35 pts	Index, proven in v24, Session 1 passed 26/05/2026
Wall Street	IX.D.DOW.IFM.IP	90%	US session 13:30–21:00 BST	50 pts	Index, US session, high volume, wider spreads
Gold	CS.D.CFDSILVER.CFD.IP	90%	London + US session 08:00–21:00 BST	40 pts	Commodity, safe haven, different volatility profile

16.2 Architecture Rules

Global position slot	Max 1 open position across ALL instruments at any time. GlobalPositionLock checked before every order submission. If locked, signal is discarded regardless of confidence.
Independent signal engines	Each instrument runs its own SignalEngine instance with its own RSI, ATR, trend state, and confidence evaluation. No shared signal state between instruments.
Independent Lightstreamer subscriptions	One LS subscription per instrument epic. Each feeds its own QuoteHub. Stale gate evaluated per instrument independently — one stale instrument does not pause others.

Per-instrument thresholds	signal_threshold defined per instrument in config. EUR/USD 88%, Japan 225 92%, Wall Street 90%, Gold 90%. All tunable without code change.
Per-instrument P&L tracking	Points engine, win rate, and P&L tracked separately per instrument AND globally. Dashboard shows both views. Helps identify which instrument contributes most and which to tune.
Per-instrument session management	Each instrument has its own maintenance window, session open/close times, and cold-start protection. EUR/USD and Gold trade near 24hr; Japan 225 and Wall Street have hard session boundaries.
Instrument activation flag	Each instrument has enabled: true/false in config. Slice 1 launches with Japan 225 only enabled. Additional instruments activated one at a time after Demo validation per instrument.
Dashboard instrument view	Web dashboard LIVE panel shows all 4 instruments simultaneously: live bid/offer, stream status, last signal confidence, and last trade result per instrument. Global position slot shown prominently — OPEN or AVAILABLE.

16.3 Activation Sequence

Slice 1 (Week 1)	Japan 225 only	Multi-instrument architecture built and proven. Japan 225 active, others defined but disabled.
Demo Week 5	+ EUR/USD	Japan 225 win rate stable at 55%+ for 2 weeks AND points state HEALTHY.
Demo Week 6	+ Wall Street	EUR/USD stable for 1 week AND combined points state still HEALTHY.
Demo Week 7	+ Gold	Wall Street stable for 1 week AND all 3 active instruments combined points HEALTHY.

17. Complete Instructions for Cursor

This is the Definitive Final Specification — v5

It supersedes ALL previous versions. Read every section before writing any code. v25 is a step change across GUI, AI/ML, and trading engine. Do not treat it as a patch to v24.

17.1 Current Codebase State

Item	Status
Branch	feature/v25-ml-enhancement
v24-stable tag	cc0d522 — original stable
v24-progui-stable tag	0f7bee2 — proven trading agent (25/05/2026)
Last Phase 1 commit	c7edf51 — all ML scaffold + display fixes
GUI fixes committed	6d5b7ac, 520eef8, 49b0eb7, 904cde1
Reconciliation script	scripts/reconcile_pending_trades.py (untracked — commit first)
E2E tests	14/14 passing
Battery	286/286 passing + soak PASS
Signal threshold	92 in config — reviewed after 20 sessions
Lightstreamer	CONNECTED:WS-STREAMING — proven working
Open market proof	PASSED — trades_opened=1 closed=1 alignment=100%
Tkinter GUI	Replaced in Slice 4 with web dashboard

17.2 Slice 1 Build Order — Week 1

Step	Build — In This Exact Sequence
0	Commit reconciliation script: git add scripts/reconcile_pending_trades.py
1	state_manager.py — all stateful components depend on this first
2	points_engine.py — everything references points state
3	environment_scorer.py — gate needed before trading can be gated
4	trade_manager.py — ATR trail, break-even, partial close, re-entry
5	session_manager.py — session end exit, gap, cold start, DST, auto-refresh

6	replay_engine.py — extends fast_replay.py
7	replay_scheduler.py — nightly cycle with 22:30 BST cutoff
8	emergency_stop.sh — must exist before Demo
9	FastAPI server skeleton — /health, /state WebSocket endpoint
10	Pull IG REST Nikkei history — run first overnight replay
11	Wire all 13 gates in trading_loop.py
12	Config validation on startup with safe defaults
13	14/14 E2E + 286/286 battery + soak — must pass before Demo enabled

17.3 Absolute Rules

- **v24 main branch NEVER touched. All work on feature/v25-ml-enhancement.**
- **14/14 E2E + 286/286 battery pass before EVERY commit — zero tolerance.**
- **Web dashboard (FastAPI + React) in Slice 4 — never extend Tkinter.**
- **USE_ML_SIGNAL = false until Slice 1 fully tested and Demo approved.**
- **Risk rules ALWAYS execute first — before fitness, points, ML gate.**
- **Trail ONLY moves in profit direction — never backwards.**
- **Session end close is mandatory — no positions through session end. Ever.**
- **Every component: try-catch, safe default, log warning, never crash.**
- **Replay labels 1x weight. Real Demo labels 3x weight. Never equal.**
- **Every model version is new file — signal_model_v1.pkl, v2, v3. Never overwrite.**
- **State files written atomically — temp file then rename. Never corrupt on crash.**
- **Replay stops 22:30 BST hard cutoff — live trading always gets full REST budget.**
- **FastAPI server NEVER writes to trading state — reads only. Zero trading path impact.**
- **Tkinter removed in Slice 4 only — after web dashboard proven over 1 week live.**
- **Report after each numbered step in Slice 1 order. Await confirmation before next.**
- **All closed trade autopsy only on IG-confirmed trades — never on pending.**
- **Deal reconciliation: always store both open_deal_id and ig_close_deal_id.**
- **reconcile_pending_trades.py runs after every session as standard practice.**